CHMI

Cattle Health Monitor

Code Walkthrough

A Developer's Guide to Architecture & Implementation

An AI-powered Streamlit web application for detecting Foot-and-Mouth Disease (FMD) and Lumpy Skin Disease (LSD) in cattle from field photographs, using a weighted ensemble of TensorFlow Lite deep-learning models.

Project	CHMI - Cattle Health Monitor
Prim	

Prepared for the development team - share internally only.

1. System Overview

CHMI is a single-page Streamlit application that lets a field user (typically a farmer or veterinary worker) photograph a cow and receive an instant AI assessment of whether the animal shows signs of Foot-and-Mouth Disease (FMD) or Lumpy Skin Disease (LSD). When disease is detected with sufficient confidence, the app surfaces the nearest veterinary contact, generates a downloadable PDF report, and records the case.

The entire application logic lives in a single module, app.py. It is organised as a linear sequence of helper functions followed by a step-driven UI controlled through Streamlit's session state. The code reads top-to-bottom: model loaders, inference utilities, OTP/SMS helpers, the report builder, geocoding helpers, and finally the four UI 'steps'.

End-to-end user journey

- **1. User Info**
  Farmer enters name, mobile, and location (auto-filled via GPS reverse-geocoding or pincode lookup).
- **2. OTP Verification**
  A 6-digit OTP is sent over SMS (SMSCountry API) to confirm the mobile number.
- **3. Dashboard**
  User enters cattle details, selects FMD or LSD, and uploads/captures an image.
- **4. Prediction & Report**
  Image is validated, run through the model ensemble, and a result + PDF report is produced.

Key design decisions

- **TensorFlow Lite over full TF**
  All models are .tflite to keep memory and load time low enough for cloud/edge deployment.
- **Weighted soft-voting ensemble**
  Five CNN architectures vote per prediction; weights favour the historically strongest models.
- **Autoencoder gatekeeper**
  A separate 'cow autoencoder' plus heuristics reject non-cattle / junk images before classification.
- **Streamlit session state machine**
  A single `st.session_state.step` string drives which screen renders, avoiding multi-page routing.

2. Project Structure

The repository is intentionally flat. Models and assets sit alongside the app for simple, path-relative loading via a single BASE_DIR anchor.

app.py	The complete application: helpers + UI state machine (~2,900 lines).

healt

Configuration anchor

Every file path is resolved relative to the module location, so the app runs identically regardless of the working directory it is launched from:

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
model_path = os.path.join(BASE_DIR, "cow_autoencoder_flex.tflite")
```

3. Model Loading & Caching

Loading TFLite interpreters is expensive, so every loader is wrapped in @st.cache_resource. This guarantees the models are read from disk and allocated exactly once per server process, then reused across every user interaction and rerun.

```
@st.cache_resource
def get_fmd_models():
    return load_tflite_models(TFLITE_MODELS_FMD)

@st.cache_resource
def get_lsd_models():
    return load_tflite_models(TFLITE_MODELS_LSD)
```

The autoencoder loader uses a special op-resolver (BUILTIN_WITHOUT_DEFAULT_DELEGATES) to avoid delegate-related crashes on some platforms. Each loader returns the interpreter together with its input/output tensor details so inference code never re-queries them.

The five-model ensemble

Both diseases use the same five architectures, each contributing a fixed weight to the final vote:

EfficientNetV2-B0 (v2b0)	weight 0.2

Effici

These weights (MODEL_WEIGHTS) were tuned so the most reliable architecture dominates while the others refine the decision.

4. Image Validation (the Gatekeeper)

Before any disease model runs, analyze_single_image() decides whether the upload is even a plausible cattle photo. This matters because the autoencoder alone is unreliable - a solid-colour or blank image can score a LOWER reconstruction error than a real cow. The function therefore layers four cheap heuristics ahead of the autoencoder.

The four-layer filter

- **1. Pixel variance**
  Rejects solid / near-solid images (var < 0.004) - real photos have rich texture.
- **2. Per-channel colour spread**
  Rejects near-grayscale images where no RGB channel has meaningful std (< 0.06).
- **3. Brightness sanity**
  Rejects pure-black or pure-white frames (mean brightness outside 0.04-0.96).
- **4. Autoencoder reconstruction error**
  Accepts only when 0.0003 <= MSE <= 0.0075: too-simple images fall below, non-cow images rise above.

```
MIN_ERROR = 0.0003
is_accepted = (MIN_ERROR <= error <= threshold)   # threshold = 0.0075
# reason: 'ok' | 'too_simple' | 'not_cow' | 'low_variance' | ...
```

The function returns a small dict ({'error', 'accepted', 'reason'}) so the UI can give the user a specific, friendly explanation when an image is rejected, rather than a generic failure.

5. The Inference Pipeline

Once an image passes validation, prediction follows a deterministic pipeline. The core is the weighted soft-voting ensemble.

Step 1 - Preprocess

Every image is opened as RGB, resized to 224x224, and scaled to float32 in [0, 1] - the exact format every model was trained on.

```
image = Image.open(uploaded_file).convert('RGB').resize((224, 224))
image_array = np.array(image).astype('float32') / 255.0
```

Step 2 - Soft-voting ensemble

soft_voting_ensemble() runs the same input through all five interpreters, normalises each model's raw output, and accumulates a weight-scaled probability vector. Two defensive helpers keep the math safe:

- **normalize_probs()**
  Falls back to a uniform distribution if a model returns an all-zero output (BUG-009 fix), preventing divide-by-zero.
- **collapse_probs_to_n()**
  Reconciles models that output a different number of classes by summing equal-sized groups, so 2-class and 4-class heads can vote together.

```
total_weighted_probs += w * probs
total_used_weight     += w
final_probs = total_weighted_probs / total_used_weight
```

Step 3 - Decision

For the 4-class FMD head, classes are folded into 'Diseased' (knuckles + mouth) vs 'Healthy' (foot + muzzle) and the larger group wins. For 2-class heads, the argmax class and its probability are returned directly. The probability becomes the confidence percentage shown to the user.

Step 4 - Confidence gating

An 80% confidence threshold governs how results are presented. Above it, the verdict is shown with full styling and (if diseased) vet contacts + PDF. Below it, the UI warns that the result is not reliable enough to act on - protecting users from low-quality captures.

6. UI State Machine

The interface is a four-state machine driven entirely by st.session_state.step. Each branch renders one screen and advances the step on success. State (OTP code, location, cattle details, upload) persists across Streamlit reruns in session_state.

user_info	Collect name, mobile, and location. GPS or pincode auto-fills village/mandal/district/state.

otp_v

Helper UI pieces

- **load_custom_css()**
  Injects the full design system (hero banners, info cards, confidence bars, styled radios).
- **render_stepper()**
  Draws the progress indicator at the top so users see where they are in the flow.
- **Location helpers**
  reverse_geocode(), fetch_pincode_data(), update_location_by_pincode(), plus fuzzy state/district matchers.

Authentication & OTP

generate_otp() uses the secrets module for a cryptographically secure 6-digit code. send_otp_sms() posts to the SMSCountry REST API using Base64 basic-auth credentials from st.secrets. When credentials are absent or placeholders, IS_DEV_MODE activates and OTP sending is simulated - so developers can run the full flow locally without an SMS account.

7. Reporting, Geolocation & Data

PDF report generation

generate_pdf_report() builds a branded one-page PDF in memory with fpdf2 and returns it as bytes for Streamlit's download button. It contains four sections - User Details, Cattle Details, Prediction Result (colour-coded red/green), and, when diseased, Recommended Veterinary Support pulled from the CSV lookups. A disclaimer footer reminds users to consult a licensed vet.

Geolocation pipeline

- **Browser GPS**
  streamlit_geolocation requests device coordinates with the user's permission.
- **Reverse geocoding**
  reverse_geocode() calls OpenStreetMap Nominatim to turn lat/lon into village/mandal/district.
- **Pincode fallback**
  fetch_pincode_data() resolves an Indian PIN code when GPS is unavailable.
- **Fuzzy matching**
  find_best_state_match() / find_best_district_match() align geocoded names to the canonical JSON list.

Reference data

Three CSVs (veternary_doctors, gopalamitra, districts) plus indian_states_districts.json drive the location cascade and the 'who to contact' recommendations. _load_csv_data() and _load_states_data() load and cache them.

Case capture

Every submitted image is written to uploaded_images/ and a matching text record (user, cattle, result, confidence) to image_metadata/, giving an auditable trail of field cases.

8. Running & Extending

Local setup

```
# macOS / Linux
bash scripts/run.sh

# Windows
commands.bat

# Verify everything is in place
python healthcheck.py
```

The run scripts create a .venv, install requirements, copy the secrets template, run the health check, and launch the app on http://localhost:8501. LSD models are optional - the app still runs FMD detection if they are missing.

Where to make common changes

Tune model trust	Edit MODEL_WEIGHTS at the top of app.py.

Adju

Notable safeguards already in place

- **Secure OTP**
  secrets.randbelow for non-guessable codes; HTML escaping on user input.
- **Zero-sum guard**
  normalize_probs() prevents divide-by-zero on degenerate model output.
- **Graceful degradation**
  Dev-mode SMS simulation and optional LSD models keep local dev frictionless.
- **Network timeouts**
  All external calls (SMS, Nominatim, pincode) use explicit timeouts.

End of walkthrough. For setup details see SETUP.txt; for a live environment check run healthcheck.py. This document reflects app.py as of the latest commit and is intended for internal developer onboarding.